



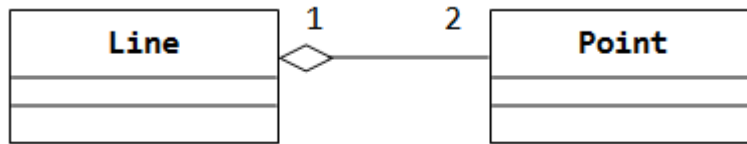
Object Oriented Software Development

Static & Dynamic Binding



Review Quizzes

► has-a relationship



```
public class Point {
    private int x;    // x co-ordinate
    private int y;    // y co-ordinate
    public int getX() {.....}
    public int getY() {.....}
    ...
}
```

```
public class Line {
    // A line composes of two points (as instance variables)
    private Point begin;    // beginning point
    private Point end;      // ending point

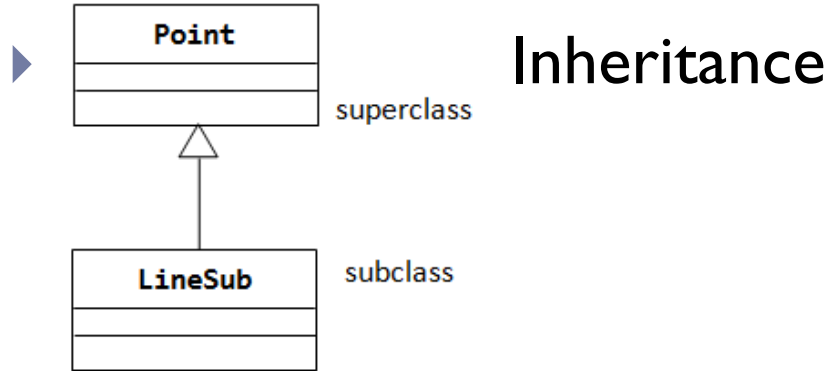
    public MyLine(int x1, int y1, int x2, int y2) {
        begin = new MyPoint(x1, y1);    // Construct the instances declared
        end   = new MyPoint(x2, y2);
    }

    public double getLength() {
        return begin.distance(end);    // Point's distance()
    }

    ...
}
```



Review Quizzes



Inheritance

```
public class Point {
    private int x;    // x co-ordinate
    private int y;    // y co-ordinate
    public int getX() {.....}
    public int getY() {.....}
    ...
}
```

```
public class LineSub extends Point {
    // A line needs two points: begin and end.
    // The begin point is inherited from its superclass Point.
    Point end;                // Ending point

    public LineSub (int beginX, int beginY, int endX, int endY) {
        super(beginX, beginY);    // construct the begin Point
        this.end = new Point(endX, endY); // construct the end Point
    }

    public double getLength() {
        return super.distance(end); // Point's distance()
    }
}
```



Agenda & Reading

► Topics:

- Static & Dynamic Binding
- Casting

► Reading

- Java how to program Late objects version (D & D)
 - Chapter 10
- The Java Tutorial
 - Creating and Using Packages
 - <http://docs.oracle.com/javase/tutorial/java/package/packages.html>
 - Controlling Access to Members of a Class
 - <http://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>
 - Overriding and Hiding Methods
 - <http://docs.oracle.com/javase/tutorial/java/landl/override.html>
 - Hiding Fields
 - <http://docs.oracle.com/javase/tutorial/java/landl/hidevariables.html>



1.Introduction

- ▶ If you have more than one method of same name (method overriding) or two variable of same name in same class hierarchy it gets tricky to find out which one is used during runtime as a result of there reference in code.
- ▶ This problem is resolved using **static and dynamic binding** in Java.
- ▶ Binding is the process used to link which **method** or **variable** to be **called** as result of the reference in code.
 - ▶ Most of the references are resolved during **compile time** but some references which depends upon **Object** and **polymorphism** in Java is resolved during **runtime** when actual object is **available**.
 - ▶ When a method call is **resolved** at **compile** time, it is known as **static binding**, while if method invocation is **resolved** at **runtime**, it is known as **Dynamic binding** or **Late binding**.



1.Introduction

- ▶ **private, final and static methods** and **variables** uses static binding and resolved by compiler because compiler knows that they can't be overridden and only possible methods are those, which are defined inside a class, whose reference variable is used to call this method.
- ▶ Static binding uses **Type information** for binding while Dynamic binding uses **Object** to resolve binding.
- ▶ Static Binding
 - ▶ Variables – static binding
 - ▶ Variables are resolved using static binding which makes there execution fast because no time is wasted to find correct method during runtime.
 - ▶ Private, final, static methods – static binding
 - ▶ Overloaded methods are resolved using static binding
- ▶ Dynamic Binding
 - ▶ Overridden methods are resolved using **dynamic binding** at runtime.



2. Static Typing Restrictions

Python

```
x = 10;  
x = "Hello"
```

Java

```
int x = 10;  
x = "Hello";
```

- ▶ Java is a “type-safe” language.

- ▶ compile-time checks are carried out to determine whether variable usage is **valid**
 - ▶ Every variable name is bounded both to a **static** type (at compile time, by means of a data **declaration**) and to an object/null. The binding to an object is optional.

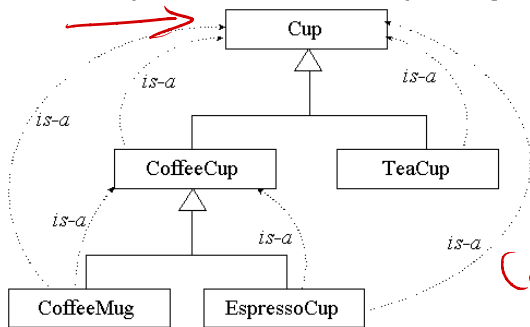
```
Cup c1 = new Cup();  
Cup c2 = null;
```

```
Cup c3;
```

- ▶ The type imposes restrictions on how a variable can be used.
 - ▶ The type restricts the classes of object to **which** the variable can refer and the **message that can be sent** using the variable

2.Static Typing Restrictions Variables

- ▶ A reference variable of static type T can refer to an instance of class T or to an instance of any of T's subclasses
- ▶ The is-a relationship applies only up the hierarchy from a subclass to its direct (and indirect) superclasses, and not down the hierarchy.



```
Cup c = new EspressoCup();  
CoffeeCup c1 = new EspressoCup();  
EspressoCup c2 = new EspressoCup();  
Cup c3 = new CoffeeMug();  
CoffeeCup c4 = new CoffeeMug();  
CoffeeMug c5 = new CoffeeMug();  
Cup c6 = new TeaCup();
```

- ▶ A superclass object cannot be treated as a subclass object, because a superclass object is **NOT** an object of any of its subclasses.
- ▶ Subclass can have additional subclass only **members**. Therefore, assigning a superclass reference to a subclass variable is not allowed

```
Person p = new Person("Dick", "Smith");  
HouseWorker w = p;
```



error: incompatible types: Person cannot be converted to HouseWorker



2. Static Typing Restrictions Messages

- ▶ Through a reference variable of static type *T*, the set of messages that can be sent using that variable are the methods defined by class *T* and its superclasses.

- ▶ For example:

Methods:
getName
cookDinner

Person

HouseWorker

Methods:
getName (inherited)
cookDinner (inherited)
cleanKitchen

```
Person p = w;  
p.getName()  
p.cookDinner()
```



```
HouseWorker w = new HouseWorker("Theresa", "Thompson");  
w.cleanKitchen();  
w.getName();  
w.cookDinner();
```



```
p.cleanKitchen();
```



Compile-time
error



3.Static Binding – Variables

Instance variables

- ▶ Static binding only uses Type information
 - ▶ Access to fields is governed by the **type** of the reference
- ▶ For example:

```
class Base {  
    public int x = 10;  
}
```

```
public class Derived extends Base {  
    public int y = 20;  
}
```

- ▶ Case 1: a Derived object with a **Derived** reference

```
//Case 1:  
Derived b1 = new Derived();  
System.out.println("b1.x=" + b1.x);  
System.out.println("b1.y=" + b1.y);
```

instance variable x : inherited from Base

instance variable y in Derived

From superclass

Derived b2 -> x=10
y=20

- ▶ Case 2: a Derived object with a **Base(superclass)** reference

```
//Case 2:  
Base b2 = new Derived();  
System.out.println("b2.x=" + b2.x);  
//System.out.println("b2.y=" + b2.y);
```

instance variable x : inherited from Base

There is no y declared in Base class
-> compile error!

Base b3 -> x=10
y=20





3.Static Binding – Variables

Two variable of same name

- ▶ Static binding only uses Type information
 - ▶ Access to fields is governed by the **type** of the reference

- ▶ For example:

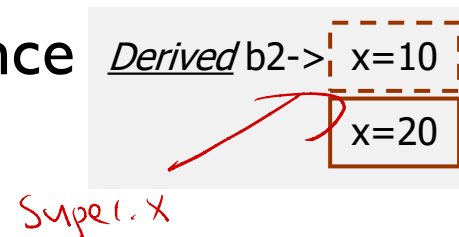
```
class Base2 {  
    public int x = 10;  
}
```

```
public class Derived2 extends Base2 {  
    public int x = 20;  
}
```

- ▶ a field that has the same name as a field in the superclass hides the superclass's field
- ▶ Case 1: a Derived object with a Derived reference

```
//Case 1:  
Derived2 b1 = new Derived2();  
System.out.println("b1.x=" + b1.x);
```

instance variable x from Derived (20)

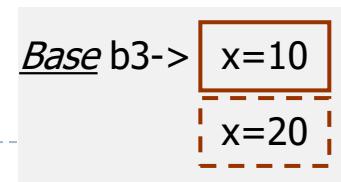


- ▶ Case 2: a Derived object with a Base(superclass) reference

b3 is declared as
type Base

```
//Case 3:  
Base2 b3 = new Derived2();  
System.out.println("b3.x=" + b3.x);
```

instance variable x from Base (10)





3.Static Binding – Variables

Get the hiding values

- ▶ Within the subclass, the field in the superclass cannot be referenced by its simple name
- ▶ To "unshadow" it by referring to `super.x`

```
public class Derived extends Base {  
    ...  
    public void method1() {  
        System.out.println("x from Derived:" + x);  
        System.out.println("x from superclass: " + super.x);  
        ...  
    }  
}
```

method In the
subclass

- ▶ Note:
 - ▶ `this.x` access the field name `x`, defined in the child class
 - ▶ `super.x` access the field name `x`, defined in a parent class
 - ▶ `super.super.x` is not valid
 - ▶ `x` was not specified as private.



3.Static Binding – Variables

Static Variables

- ▶ Static binding only uses Type information
 - ▶ Access to fields is governed by the type of the reference
 - ▶ Class variable: one copy only, all instances of the class share the same static variable

▶ Example:

```
class Base3 {  
    public static int x=10;  
}
```

Invoke the static variable in
the Base3 class

```
System.out.println("Base3.x=" + Base3.x);
```

```
System.out.println("Derived3.y=" + Derived3.y);  
System.out.println("Derived3.y=" + Derived3.x);
```

Invoke the static variables in
the Derived3 class

```
public class Derived3 extends Base3 {  
    public static int y=20;  
}
```

class variables:
one copy only

Base: x= 10
Derived: y=20

Base3.x=10
Derived3.y=20
Derived3.y=10



3.Static Binding

final and static methods

- ▶ A final method in a superclass cannot be overridden in a subclass.
- ▶ A final method's declaration can never change, so all subclasses use the **same method implementation**, and calls to final methods are resolved at compile time—this is known as static binding.

```
class Base6 {  
    public final void get() { System.out.println("final:Base:get"); }  
}  
public class Derived6 extends Base6 {  
    /*  
        public void get() {  
            System.out.println("final:Derived:get");  
        }  
    */  
}
```

error: get() in Derived6 cannot override get() in Base6



3.Static Binding static Methods

- ▶ Only the non-static methods of a class can be overridden

- ▶ Example:

```
class Base4 {  
    public static int x = 10;  
    public static void get() {  
        System.out.println("static:Base:get");  
    }  
}
```

public int x = 10

```
public class Derived4 extends Base4 {  
    public static int x = 20;  
    public static void get() {  
        System.out.println("static:Derived:get");  
    }  
}
```

Invoke the static method in
the Base4 class

`Base4.get();`

`Derived4.get();`

`static:Base:get`
`static:Derived:get`

Invoke the static method in
the Derived4 class



3.Static Binding

Overloading Methods

- ▶ The practice of defining two or more methods **within the same class** that share the **same** name but have different signatures is called overloading methods.
- ▶ The signature of a method is comprised of its name, its parameter types and the order of its parameters.
- ▶ The signature of a method is not comprised of its return type nor its visibility.

```
class Base7 {  
    public void aMethod() {  
        System.out.println("called aMethod()");  
    }  
    public void aMethod(int x) {  
        System.out.println("called aMethod(x)");  
    }  
    public static void main(String[] args) {  
        Base7 b1 = new Base7();  
        b1.aMethod(8);  
    }  
}
```

called aMethod(x)



4. Dynamic Binding

- ▶ Dynamic Binding refers to the case where compiler is not able to resolve the call and the binding is done at runtime only.
 - ▶ Java compiler can't be sure of what type of object this reference would be pointing to at runtime.
 - ▶ Polymorphism in Java is resolved during runtime when actual object is available.

- ▶ Example:

```
class Base8 {  
    public void g() {  
        System.out.println("Base:g");  
    }  
}
```

```
public class Derived8 extends Base8 {  
    public void g() {  
        System.out.println("Derived:g");  
    }  
}
```

- ▶ Case 1: a Derived object with a Derived reference

```
//Case 1:  
Derived8 b1 = new Derived8();  
b1.g();
```

Derived:g

g() in Derived? ✓
Invoke g() in Derived

① compile time
Can we find the method?
② run time

- ▶ Case 2: a Derived object with a Base(superclass) reference

```
//Case 3:  
Base8 b2 = new Derived8();  
b2.g();
```

Derived:g

g() in Base? ✓
Invoke g() in Derived



Exercise 1

- ▶ Given the following classes definitions:

- ▶ Objects:

```
Foo a = new Foo();  
Bar b = new Bar();
```

```
class Foo {  
    private int i;  
    public void f() { System.out.println("Foo:f"); }  
    public void g() { System.out.println("Foo:g"); }  
}  
class Bar extends Foo {  
    public int j;  
    public void g() { System.out.println("Bar:g"); }  
}
```

- ▶ what is the output?

```
b.f(); Foo:f  
b.g(); Bar:g  
a.g(); Foo:g  
a.f(); Foo:f
```

- ▶ legal or compile-time error?

```
b.j = 3;  
a.j = 3;  
  
b.i = 3;  
a.i = 3;
```

A
-
B
-
C



Exercise 2

- ▶ Given the following classes definitions:

```
A ref1 = new A();  
A ref2 = new C();  
B ref3 = new B();  
C ref4 = new C();
```

- ▶ objects:

- ▶ compile-time error?

```
System.out.println(ref1.f()); 0  
System.out.println(ref1.g()); 3
```

```
System.out.println(ref2.f()); 2  
System.out.println(ref2.g());
```

```
System.out.println(ref3.f()); 1  
System.out.println(ref3.g()); 1
```

```
System.out.println(ref4.f()); 2  
System.out.println(ref4.g()); 2
```

```
class A {  
    public int f() { return 0; }  
    public int g() { return 3; }  
}  
class B extends A {  
    public int f() { return 1; }  
    public int g() { return f(); }  
}  
class C extends B {  
    public int f() { return 2; }  
}
```

Handwritten notes:
for in A2
gin A2
→ gin B → go back to C look for f() → 2

- ▶ If no error, what is the output?



Exercise 3

► Given the following classes definitions:

► objects:

```
ClassA obj1 = new ClassA();  
ClassB obj2 = new ClassB();
```

```
class ClassA {  
    public ClassA() {}  
    public int method1(double a){return 0;}  
    public double method2(int b){ return 0.5; }  
}  
class ClassB extends ClassA {  
    public ClassB(){}  
    public int method1(int b, int c){ return 1; }  
    public double method2(double c){return 1.5;}  
}
```

► compile-time error?

```
System.out.println(obj2.method2(3)); 0.5  
System.out.println(obj2.method2(3.5)); 1.5
```

```
System.out.println(obj1.method2(3)); 0.5  
System.out.println(obj1.method2(3.5)); 1.0
```

```
System.out.println(obj1.method1(3)); 0  
System.out.println(obj1.method1(3.5)); 0
```

```
System.out.println(obj2.method1(3, 5)); 1  
System.out.println(obj2.method1(3)); 0  
System.out.println(obj2.method1(3.5, 5)); 1.0
```



5. Casting

- ▶ **Casting** an object to a different type will **permit** access to fields shadowed by the type of the original object
- ▶ Casting an object to a different type will have **NO** effect on which method is invoked in response to a given message. Once overridden, methods remain overridden
- ▶ To cast, just precede a value with the parenthesised name of the desired type
 - ▶ A **ClassCastException** is thrown if the type cast is incompatible with the type of object to be pointed to.
 - ▶ To check that a potential cast is safe, the `instanceof` operator should be used
 - ▶ The **`instanceof`** operator Can be used to determine the type of an object
 - ▶ Returns **`true`** if `<obj>` can be assigned into a reference of type `<Class name>`. That is, `instanceof` checks if the `<obj>` is-an **instance** of the specified class, provided `obj` is not null.

```
<obj> instanceof <Class name>
```



5. Casting Primitive Types

► Widening conversions

- Wider assignment
- Wider Casting
 - Casting a value to a wider value is always permitted but never required. However, explicitly casting can make your code more readable

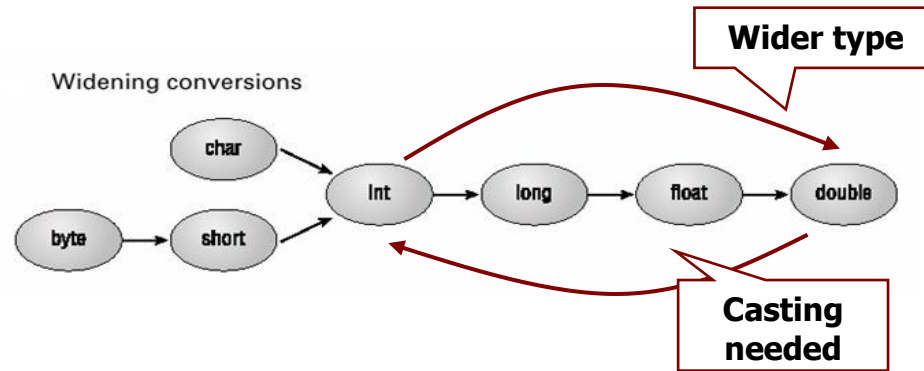
```
double d = 4.9;  
int i = 10;  
double d1, d2;
```

```
d1 = i;
```

Assignment conversion
d1 = 10.0

```
d2 = (double) i;
```

**Casting conversion
(optional)**
d2 = 10.0



► Narrowing conversion

- Narrow assignment – Compile-time error
- Narrow Casting – Loss of information
 - You must use an explicit cast to convert a value in a large type to a smaller type, or else converting that value might result in a loss of precision.

```
double d = 4.9;  
int i = 10;  
int i1, i2;
```

```
//i1 = d;
```

Assignment conversion
Compile-time error

```
i2 = (int) d;
```

Narrow Casting: i2=4
NOTE: Everything beyond the decimal point will be truncated

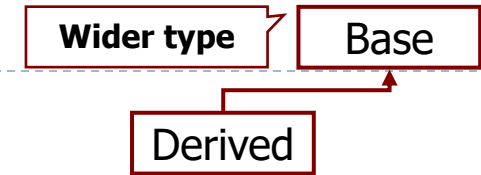
Illegal conversion of a quart to a cup, with loss of data





5. Casting Object Types

Example: casting/CastObject.java



► Widening conversions

- Wider object reference assignment conversion
- Wider object reference casting (optional)

```
Base b = new Base();
Derived d = new Derived();
Base b1, b2;
System.out.println(d.y);
```

```
b1 = d;
//System.out.println(b1.y);
```

Assignment conversion - OK
However, no access to fields in Derived

```
b2 = (Base) d;
//System.out.println(b2.y);
```

Widening Casting conversion - OK
However, no access to fields in Derived

► Note:

- Reference conversion, like primitive conversion, takes place at compile time, because the compiler has all the information it needs to determine whether the conversion is legal
- Inheritance relation is an “is a” relation; the parent object is more general than child object
- The general rule of thumb is that converting to a superclass is permitted (because super class is wider than sub class)



5. Casting Object Types

Wider type

Person

Student

Example: casting/CastObject.java

► Narrowing conversion

- Narrow object reference assignment – Compile-time error
- Narrow object reference casting –
 - Compile-time OK,
 - Run-time?
 - Like primitive casting: By using a cast, you convince the compiler to let you do a conversion that otherwise might not be allowed
 - The run-time check determines whether the class of the object being cast is compatible with the new type

```
Base b = new Base();
Derived d = new Derived();

Derived d1, d2, d3;
```

```
//d1 = b;
```

Compile-time error

Base b -> x=10

```
d2 = (Derived) b;
```

Compile-time OK, Run-time ERROR
b is an instance of class Base, not Derived

java.lang.ClassCastException: Base

b is instance of derived!!

```
Base d_as_b = new Derived();
```

```
d3 = (Derived) d_as_b;
```

Compile-time OK, narrow casting,
casting from superclass to subclass

Run-time OK
d_as_b is an instance of Derived

Base d_as_b -> x=10

y=20

*b2.y x ✓
b2.x ✓
→ ((Derived) b2).y*



Using Casting

► Shadowing

- Using type casting, it is possible to recover a hidden member (class variable, class method, or an instance variable), but not an overridden method (i.e. an instance method). An overridden method can be recovered only by means of a super call in derived class

```
class Base2 {  
    public int x = 10;  
}
```

```
public class Derived2 extends Base2 {  
    public int x = 20;  
}
```

```
Derived2 d = new Derived2();  
Base2 b = new Derived2();  
  
System.out.println( "d.x=" + d.x);  
System.out.println( "(Base2)d .x=" + ((Base2)d).x);
```

20
10

```
System.out.println( "b.x=" + b.x);  
System.out.println( "(Derived2)b .x=" + ((Derived2)b).x);
```

10
20

Derived2 d->

x=10

x=20

Hiding

Base2 b->

x=10

x=20

Hiding



Exercise 4

- ▶ Consider the Person & HouseWorker classes defined previously:

```
Person hw3 = new HouseWorker("Theresa", "Thompson");  
hw3.cleanKitchen(); //compile-time error
```

HouseWorker object with reference to a person.

as cleanKitchen is defined in the HouseWorker class, but not in Person class

- ▶ How can you invoke the cleanKitchen()?

- ▶ Consider the Base & Derived classes defined previously:

```
Base b2 = new Derived();  
  
b2.method2() //compile-time error
```

```
class Base {  
    ...  
    public void method1() ...  
}  
class Derived extends Base {  
    ...  
    public void method2() ...  
}
```

- ▶ How can you invoke the method2()?



Exercise 5

- Consider the ClassA & ClassB defined previously:

```
ClassA obj3 = new ClassB();
```

*ClassB object with reference
to classA*

```
class ClassA {  
    public ClassA() {}  
    public int method1(double a){return 0;}  
    public double method2(int b){ return 0.5; }  
}  
  
class ClassB extends ClassA {  
    public ClassB(){}  
    public int method1(int b, int c){ return 1; }  
    public double method2(double c){return 1.5;}  
}
```

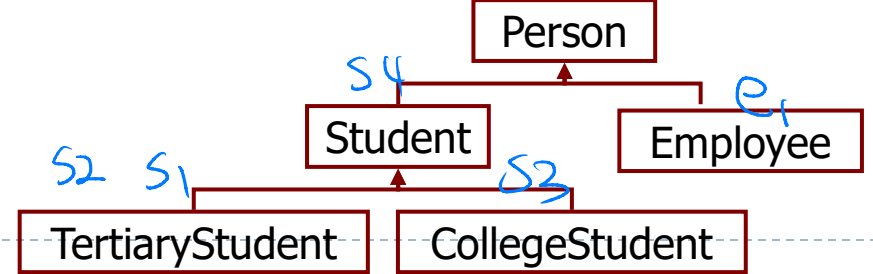
```
System.out.println(obj3.method2(3.5)); //compile-time error
```

```
System.out.println(obj3.method1(3, 6)); //compile-time error
```

- How can you invoke the above methods()?



5.Casting Rules



- ▶ At compile-time, checking is carried out to determine whether the cast-type and the type of the existing reference variable are related through inheritance.
- ▶ At run time, a check is made to determine whether the object actually pointed to is compatible with the cast-type.

```
Student s1 = new TertiaryStudent();
TertiaryStudent s2 = new TertiaryStudent();

Employee e1 = new Employee();

Student s3 = new CollegeStudent();
Student s4 = new Student();
```

```
ts = (TertiaryStudent) s3;
```

run-time error as s3 is an object of CollegeStudent

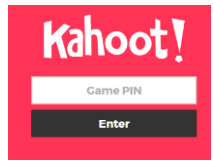
```
ts = (TertiaryStudent) s4;
```

run-time error as s4 is an object of Student

```
if (s3 instanceof TertiaryStudent)
    ts = (TertiaryStudent) s3;
```

No error message at all

	Compile-time	Run-time
(TertiaryStudent) s1	OK	OK
(TertiaryStudent) s2	OK	OK
(TertiaryStudent) e1	ERROR	
(TertiaryStudent) s3	OK <i>As it is referenced as the Student object</i>	ERROR <i>As it is actually a college student</i>
(TertiaryStudent) s4	OK	ERROR <i>Student.</i>
(Students) S2	OK	OK



Summary

```
Student s = new Student();  
Person p = s;
```

► Object Conversion

- Make it possible to reference an object in a different way by assigning it to an object reference of a different type.
- It is handled automatically by the compiler.
- The object is NOT converted or changed in any way. Once instantiated, an object NEVER changes its type (a Cat will always be a Cat). The concept is similar to the "widening conversions" performed on primitive data types.

► Casting

- It is needed when the compiler doesn't recognize an automatic conversion. The concept is similar to the casting of primitive data types to perform a "narrowing conversion".
- Casting does not change the reference or the object being pointed to. It only changes the compiler's treatment of the reference
- Casting is only legal between objects in the same inheritance hierarchy
- You can safely cast from an object to its superclass



Overriding, hiding, and overloading methods

- ▶ An instance method in a subclass with the same signature (name, plus the number and the type of its parameters) and return type as an instance method in the superclass **overrides** the superclass's method.
- ▶ If a subclass defines a class method with the same signature as a class method in the superclass, the method in the subclass **hides** the one in the superclass.
 - ▶ The distinction between hiding and overriding has important implications.
 - ▶ The version of the overridden method that gets invoked is the one in the subclass.
 - ▶ The version of the hidden method that gets invoked depends on whether it is invoked from the superclass or the subclass.
- ▶ **Overloaded** methods are differentiated by the number and the type of the arguments passed into the method.
 - ▶ The compiler does not consider return type when differentiating methods, so you cannot declare two methods [in the same class] with the same signature even if they have a different return type.



Exercise 6



- ▶ Consider the class definitions:
- ▶ Objects:

```
Base01 b = new Base01();
```

```
Base01 c = new Derived01();
```

x=10 y=11

x2=20 y2=21

- ▶ What is the value of ...
 - ▶ b.x, b.y, b.foo(), b.goo()
 - ▶ c.x, c.y, c.foo(), c.goo()
 - ▶ c.x2, c.foo2() //compile time error
 - ▶ How to get c.x2 and c.foo2()?

```
class Base01 {  
    public int x = 10;  
    static int y = 10;  
    Base01() {  
        x = y++;  
    }  
    public int foo() {  
        return x;  
    }  
    public static int goo() {  
        return y;  
    }  
}
```

```
class Derived01 extends Base01 {  
    int x2 = 20;  
    static int y2 = 20;  
    Derived01() {  
        x2 = y2++;  
    }  
    public int foo2() {  
        return x2;  
    }  
    public static int goo2() {  
        return y2;  
    }  
}
```



Exercise 7



- ▶ Consider the class definitions:
- ▶ Objects:

```
Base02 b = new Base02();  
Base02 c = new Derived02();  
Derived02 d = new Derived02();
```

- ▶ What is the value of ...
 - ▶ b.x, b.foo()
 - ▶ c.x, c.foo()
 - ▶ d.x, d.foo()
 - ▶ ((Derived02)c).x
 - ▶ ((Derived02)c).foo()

```
class Base02 {  
    public int x = 10;  
    static int y = 10;  
    Base02() {  
        x = y++;  
    }  
    public int foo() {  
        return x;  
    }  
    public static int goo() {  
        return y;  
    }  
}
```

```
class Derived02 extends Base02 {  
    int x= 20;    // shadow  
    static int y = 20;  
    Derived02() {  
        x = y++;  
    }  
  
    public int foo() { // overriding  
        return x;  
    }  
    public static int goo() {  
        return y;  
    }  
}
```


Exercise 8

- Consider the following class definitions:

```
class Base04 {  
    int i = 1;  
    public int f() {  
        return i;  
    }  
}
```

```
class Derived04 extends Base04 {  
    int i = 2; // shadows i in Base04  
    public int f() { // override f() in Base04  
        return -i;  
    }  
}
```

- What is the output?

```
Derived04 b = new Derived04();  
Base04 a = (Base04) b; // casting b to Base04's instance  
System.out.println( a.i ); // refer to Base04.i  
System.out.println( a.f() );  
System.out.println( b.i );  
System.out.println( b.f() );
```



Exercise 9

- ▶ A call of an overridden method can only be resolved at runtime as the compiler can't be sure of what type of object this reference would be pointing to at runtime.
- ▶ The search for a method begins with the dynamic class.
 - ▶ If this class doesn't implement the method (i.e. it doesn't introduce or override the method), the search progresses to its superclass. This process is repeated up the hierarchy until the method is found.

```
Derived b2 = new Derived();  
Base b3 = new Derived();
```

```
class Base {  
    public void f() {}  
    public void g() {}  
}
```

```
public class Derived extends Base {  
    public void g() { ... }  
    public void h() {...}  
}
```

Dynamic class

b2.f();

f() in Derived, f() in Base?
Invoke f() in ____

b2.g();

g() in Derived?
Invoke g() in ____

b2.h();

h() in Derived?
Invoke h() in ____

b3.f();

f() in Base?
Invoke f() in ____

b3.g();

g() in Base?
Invoke g() in ____

b3.h();

h() in Base?
Compile time error